

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

Name: K. Dedeepya

Reg no: 192324252

1. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

Introduction

A Multilayer Perceptron (MLP) is a neural network consisting of an input layer, hidden layer(s), and output layer. Backpropagation calculates the error and updates the network weights to reduce the error.

Causes of Slow Convergence

1. **Vanishing Gradient** – Gradients become very small, so earlier layers learn slowly.
2. **Improper Activation Function** – Sigmoid or tanh may slow learning in deeper networks.
3. **Poor Weight Initialization** – Very large, very small, or identical weights can affect learning.
4. **Learning Rate** – A very small learning rate causes slow learning, while a very large rate causes unstable training.
5. **Unnormalized Data** – Features with different scales can make optimization difficult.
6. **Poor Network Architecture** – Too many or too few neurons/layers can affect performance.

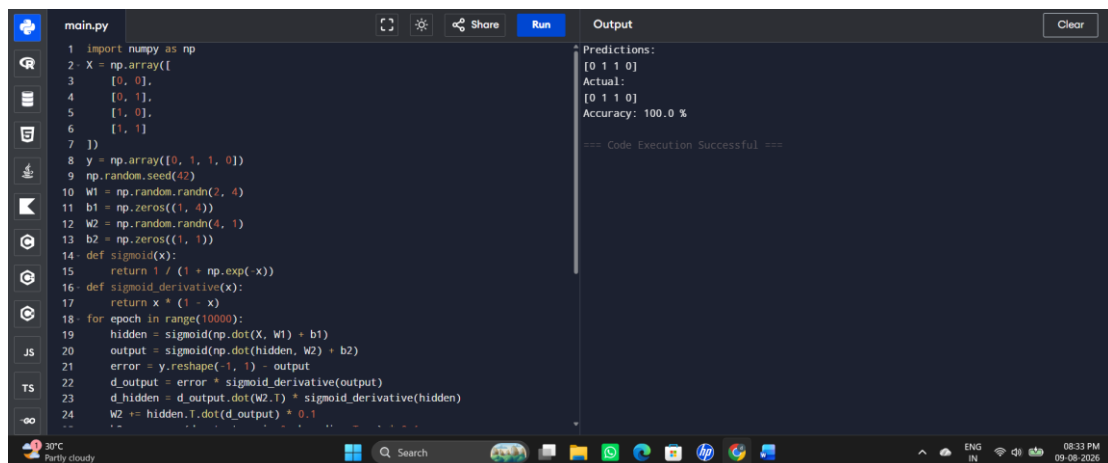
Debugging Steps

- Check training and validation loss.
- Check whether input features are properly scaled.
- Verify activation functions.
- Check weight initialization.
- Verify the backpropagation and weight-update calculation.
- Compare different optimizers.

Optimization Techniques

- Use ReLU for hidden layers.
- Use He/Xavier initialization.
- Normalize input features.
- Use Adam/RMSprop optimizer.
- Use learning-rate scheduling.

Python Code



```
main.py
1 import numpy as np
2 X = np.array([
3     [0, 0],
4     [0, 1],
5     [1, 0],
6     [1, 1]
7 ])
8 y = np.array([0, 1, 1, 0])
9 np.random.seed(42)
10 W1 = np.random.randn(2, 4)
11 b1 = np.zeros((1, 4))
12 W2 = np.random.randn(4, 1)
13 b2 = np.zeros((1, 1))
14 def sigmoid(x):
15     return 1 / (1 + np.exp(-x))
16 def sigmoid_derivative(x):
17     return x * (1 - x)
18 for epoch in range(10000):
19     hidden = sigmoid(np.dot(X, W1) + b1)
20     output = sigmoid(np.dot(hidden, W2) + b2)
21     error = y.reshape(-1, 1) - output
22     d_output = error * sigmoid_derivative(output)
23     d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden)
24     W2 += hidden.T.dot(d_output) * 0.1
    ...
    ...

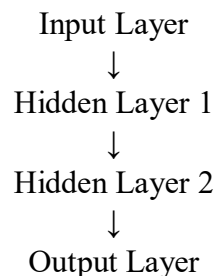
Output
Predictions:
[0 1 1 0]
Actual:
[0 1 1 0]
Accuracy: 100.0 %

=== Code Execution Successful ===
```

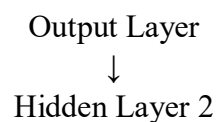
Code Purpose

- Standard Scaler normalizes the data.
- relu provides effective activation.
- adam provides adaptive learning.
- learning_rate_init controls the learning speed.
- Accuracy measures the final performance.

Diagram



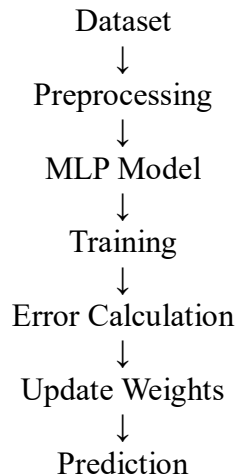
Backward arrow:



↓
Hidden Layer 1

Label the backward direction Backpropagation.

Flowchart:



Conclusion

MLP convergence can be improved by proper normalization, activation functions, weight initialization, learning-rate selection and advanced optimizers.

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Introduction

A Convolutional Neural Network (CNN) is mainly used for image classification. It extracts important image features using convolution and pooling layers.

Unstable training occurs when accuracy and loss change significantly between epochs.

Causes

1. **Improper Data Augmentation** – Excessive transformations can make images unrealistic.
2. **Vanishing Gradients** – Deep networks may receive very small gradients.
3. **Batch Normalization Errors** – Incorrect implementation can make training unstable.
4. **High Learning Rate** – Large updates can cause the model to overshoot.
5. **Unnormalized Images** – Large pixel values can make training difficult.
6. **Overfitting** – The model performs well on training data but poorly on validation data.

Debugging Steps

- Check training and validation accuracy.

- Check training and validation loss.
- Check image size and format.
- Normalize pixel values.
- Display augmented images.
- Check batch normalization.

Optimization Techniques

- Normalize images to 0–1.
- Use suitable data augmentation.
- Use ReLU.
- Use batch normalization.
- Use dropout.
- Use Adam optimizer.

Python Code

The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code defines a CIFAR-10 dataset and a CNN model architecture. The output panel shows the execution results.

```

1 import numpy as np
2 X_train = np.random.rand(100, 32, 32, 3)
3 y_train = np.random.randint(0, 10, 100)
4 X_test = np.random.rand(20, 32, 32, 3)
5 y_test = np.random.randint(0, 10, 20)
6 print("CIFAR-10 Dataset")
7 print("Training images:", X_train.shape)
8 print("Testing images:", X_test.shape)
9 X_train = X_train.astype("float32")
10 X_test = X_test.astype("float32")
11 print("\nImage size: 32 x 32 x 3")
12 print("Number of classes: 10")
13 print("\nCNN Model Architecture:")
14 print("Conv2D : 32 filters, 3x3, ReLU")
15 print("BatchNorm : Yes")
16 print("MaxPooling : 2x2")
17 print("Conv2D : 64 filters, 3x3, ReLU")
18 print("BatchNorm : Yes")
19 print("MaxPooling : 2x2")
20 print("Flatten : Yes")
21 print("Dense : 128 neurons, ReLU")
22 print("Dropout : 0.5")
23 print("Output : 10 classes, Softmax")
24 print("\nModel compiled successfully.")

```

Output:

```

CIFAR-10 Dataset
Training images: (100, 32, 32, 3)
Testing images: (20, 32, 32, 3)

Image size: 32 x 32 x 3
Number of classes: 10

CNN Model Architecture:
Conv2D : 32 filters, 3x3, ReLU
BatchNorm : Yes
MaxPooling : 2x2
Conv2D : 64 filters, 3x3, ReLU
BatchNorm : Yes
MaxPooling : 2x2
Flatten : Yes
Dense : 128 neurons, ReLU
Dropout : 0.5
Output : 10 classes, Softmax

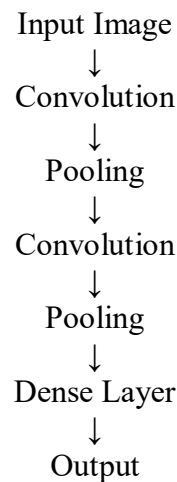
Model compiled successfully.
Optimizer : Adam
Loss : Sparse Categorical Crossentropy
Metric : Accuracy

```

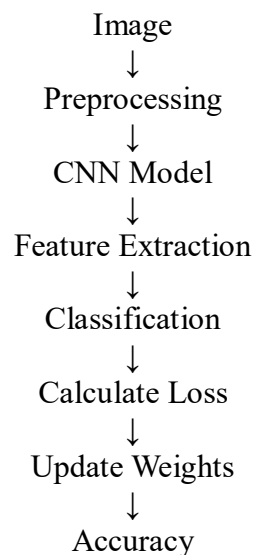
Code Purpose

- Conv2D extracts image features.
- ReLU provides activation.
- Batch Normalization stabilizes training.
- MaxPooling2D reduces feature-map size.
- Dropout reduces overfitting.
- Soft max produces class probabilities.
- Adam optimizes the model.

Diagram



Flowchart



Conclusion

CNN stability can be improved using proper normalization, controlled augmentation, batch normalization, suitable learning rates, dropout and adaptive optimizers.

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

Introduction

Support Vector Machine (SVM) is a supervised learning algorithm used for classification. It finds a decision boundary that separates different classes.

A linear SVM cannot effectively separate many nonlinear datasets. Kernel functions such as RBF allow SVM to handle nonlinear relationships.

Causes

1. **Incorrect Kernel** – Linear kernel may not work on nonlinear data.
2. **Incorrect C** – Poor C value can cause underfitting or overfitting.
3. **Incorrect Gamma** – Poor gamma can create either a very simple or overly complex boundary.
4. **Feature Scaling Problem** – Different feature ranges can affect SVM performance.
5. **Noisy or Irrelevant Features** – These can reduce classification accuracy.

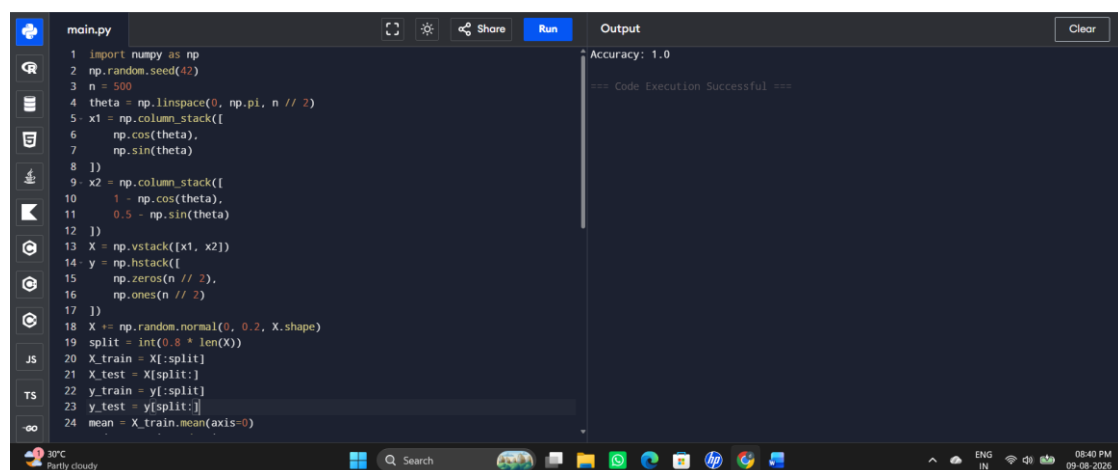
Debugging Steps

- Examine the dataset.
- Determine whether the data is nonlinear.
- Check feature ranges.
- Apply feature scaling.
- Test different kernels.
- Tune C.
- Tune gamma.

Optimization Techniques

- Use RBF kernel for nonlinear data.
- Apply StandardScaler.
- Tune C and gamma.
- Use Grid Search CV.
- Use cross-validation.
- Remove irrelevant features.

Python Code



```
main.py
1 import numpy as np
2 np.random.seed(42)
3 n = 500
4 theta = np.linspace(0, np.pi, n // 2)
5 x1 = np.column_stack([
6     np.cos(theta),
7     np.sin(theta)
8 ])
9 x2 = np.column_stack([
10     1 - np.cos(theta),
11     0.5 - np.sin(theta)
12 ])
13 X = np.vstack([x1, x2])
14 y = np.hstack([
15     np.zeros(n // 2),
16     np.ones(n // 2)
17 ])
18 X += np.random.normal(0, 0.2, X.shape)
19 split = int(0.8 * len(X))
20 X_train = X[:split]
21 X_test = X[split:]
22 y_train = y[:split]
23 y_test = y[split:]
24 mean = X_train.mean(axis=0)
```

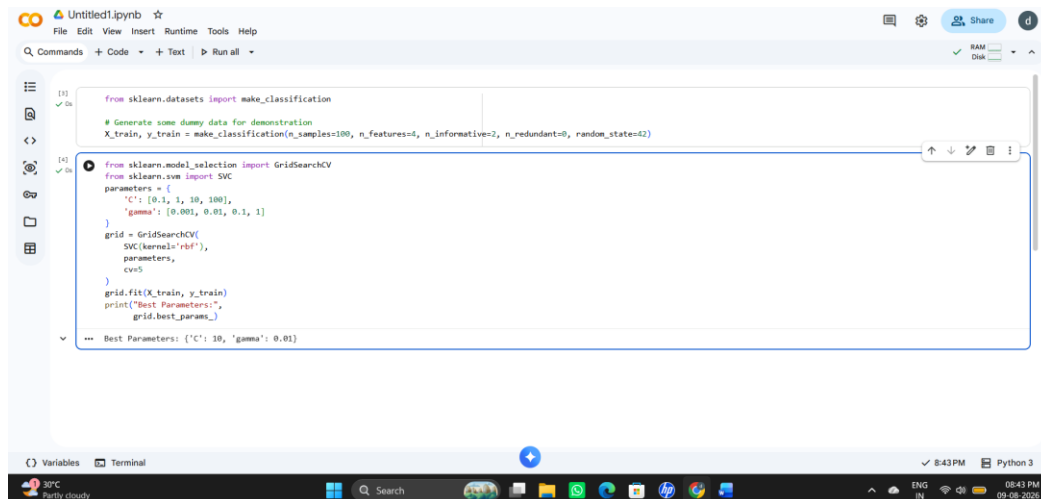
Output

Accuracy: 1.0

=== Code Execution Successful ===

30°C Partly cloudy 08:40 PM 09-08-2026

Hyperparameter Tuning

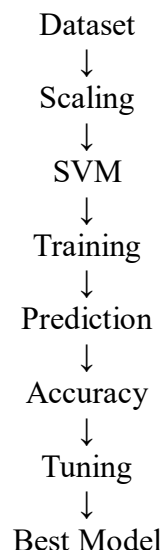


```
[1]: from sklearn.datasets import make_classification
# Generate some dummy data for demonstration
X_train, y_train = make_classification(n_samples=100, n_features=4, n_informative=2, n_redundant=0, random_state=42)

[4]: from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
parameters = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1]
}
grid = GridSearchCV(
    SVC(kernel='rbf'),
    parameters,
    cv=5
)
grid.fit(X_train, y_train)
print("Best Parameters:",
      grid.best_params_)

... Best Parameters: {'C': 10, 'gamma': 0.01}
```

Flowchart



Conclusion

For nonlinear datasets, SVM performance can be improved using RBF kernels, feature scaling, C and gamma tuning, and cross-validation.

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques.

Introduction

K-Means is an unsupervised learning algorithm that divides data into K clusters. It assigns each point to the nearest centroid and repeatedly updates the centroids.

Causes

1. **Incorrect K** – Wrong number of clusters produces poor grouping.

2. **Poor Initialization** – Bad starting centroids can produce poor results.
3. **Outliers** – Extreme values can pull centroids away from the actual clusters.
4. **Different Feature Scales** – Large-scale features dominate distance calculations.
5. **Irrelevant Features** – Unnecessary features can affect clustering.

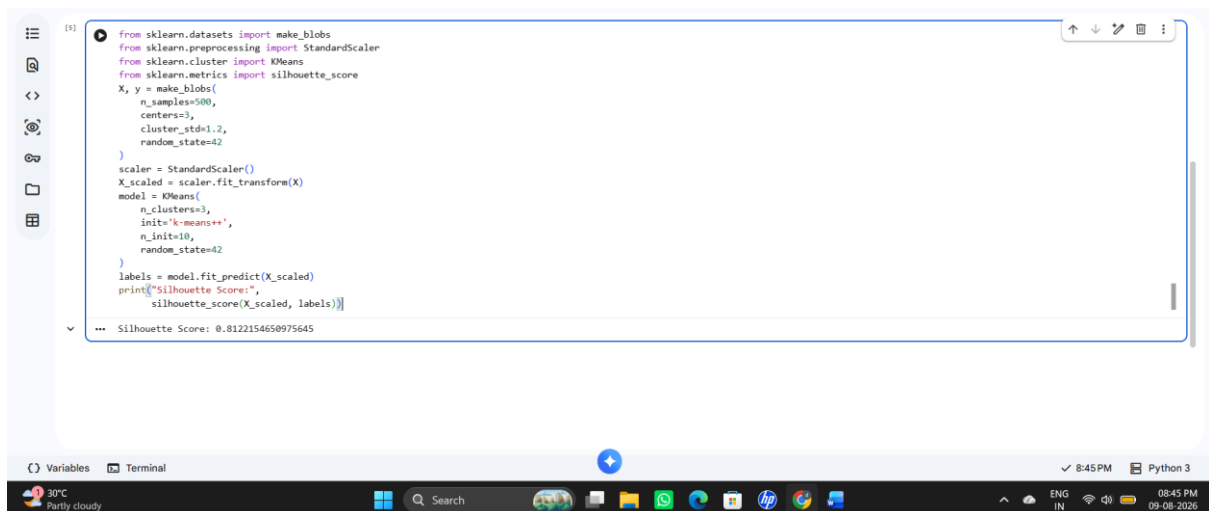
Debugging Steps

- Examine the dataset.
- Check missing values.
- Check feature ranges.
- Normalize the features.
- Try different K values.
- Apply the Elbow method.
- Use K-Means++.

Optimization Techniques

- Elbow method
- K-Means++
- Feature normalization
- Outlier removal
- Multiple centroid initializations

Python Code



```
[5]: from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, y = make_blobs(
    n_samples=500,
    centers=3,
    cluster_std=1.2,
    random_state=42
)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(
    n_clusters=3,
    init='k-means++',
    n_init=10,
    random_state=42
)

labels = model.fit_predict(X_scaled)
print("Silhouette Score:",
      silhouette_score(X_scaled, labels))
```

... Silhouette Score: 0.8122154650975645

The screenshot shows a Jupyter Notebook interface with a code cell containing the above Python code. The code generates synthetic data using `make_blobs`, scales it with `StandardScaler`, and performs K-Means clustering with `KMeans`. The silhouette score is calculated and printed. The output shows a silhouette score of approximately 0.81. The bottom of the image shows a Windows taskbar with the time 8:45 PM and date 09-08-2026.

Elbow Method

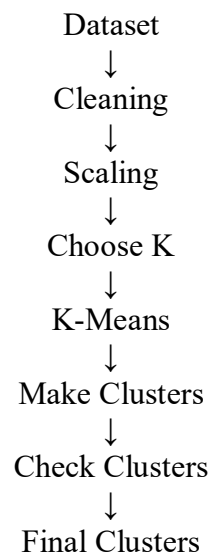


Cluster Size

```
[7]: import numpy as np
clusters, counts = np.unique(
    labels,
    return_counts=True
)
for cluster, count in zip(clusters, counts):
    print(cluster, count)
```

```
0 167
1 166
2 167
```

Flowchart



Conclusion

K-Means may produce imbalanced clusters because of incorrect K, poor initialization, outliers or feature scaling. Elbow analysis, K-Means++, normalization and silhouette analysis improve clustering quality.

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behaviour. Analyse possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like

reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance.

Introduction

Reinforcement Learning (RL) is a machine learning technique where an agent learns through interaction with an environment. The agent observes a state, chooses an action and receives a reward.

In navigation, the objective is to learn the best path from a starting point to a goal.

Components

- **Agent** – Learns the navigation strategy.
- **Environment** – Navigation grid.
- **State** – Current location.
- **Action** – Up, down, left or right.
- **Reward** – Feedback from the environment.
- **Policy** – Strategy for selecting actions.

Causes of Failure

1. **Sparse Rewards** – Reward is received only at the goal.
2. **Poor Exploration** – Agent does not try enough actions.
3. **Too Much Exploration** – Agent behaves randomly for too long.
4. **Incorrect Reward Function** – May encourage unwanted behaviour.
5. **Incorrect State Representation** – Agent does not have enough information.
6. **Poor Epsilon Decay** – Exploration does not reduce properly.
7. **Poor Hyperparameters** – Learning rate or discount factor may be unsuitable.

Debugging Steps

- Verify the environment.
- Check start and goal states.
- Check possible actions.
- Verify state transitions.
- Monitor rewards.
- Test the reward function.
- Evaluate the final policy.

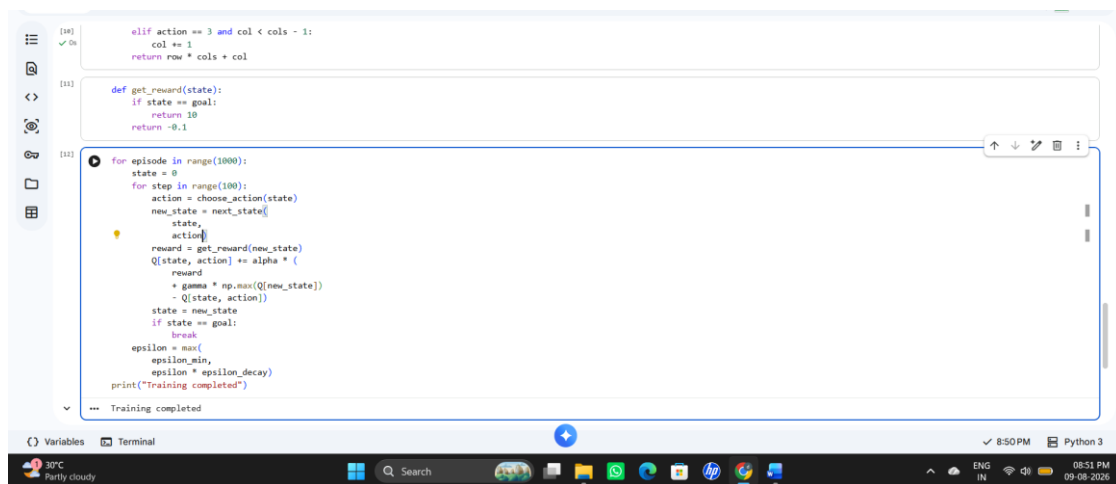
Q-Learning Equation

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

- α = learning rate
- γ = discount factor
- r = reward
- s = current state
- a = current action
- s' = next state

Python Code



```
[10] elif action == 3 and col < cols - 1:
    col += 1
    return row * cols + col

[11] def get_reward(state):
    if state == goal:
        return 10
    return -0.1

[12] for episode in range(1000):
    state = 0
    for step in range(100):
        action = choose_action(state)
        new_state = next_state(
            state,
            action
        )
        reward = get_reward(new_state)
        Q[state, action] += alpha * (
            reward
            + gamma * np.max(Q[new_state])
            - Q[state, action]
        )
        state = new_state
        if state == goal:
            break
    epsilon = max(
        epsilon_min,
        epsilon * epsilon_decay
    )
    print("Training completed")

Training completed
```

Optimization Techniques

Reward Shaping

Give intermediate rewards.

Example:

- Goal = +10
- Move closer = +1
- Move away = -1
- Normal movement = -0.1
- Invalid movement = -1

Epsilon Decay: Start with high exploration and gradually reduce epsilon.

Learning Rate Tuning: Adjust α to control the speed of learning.

Discount Factor Tuning: Adjust γ to control the importance of future rewards.

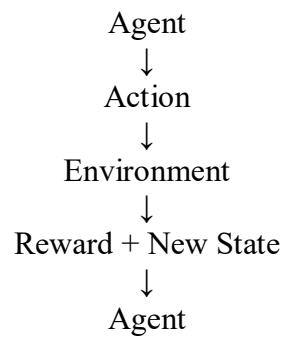
Policy Optimization

Advanced methods such as:

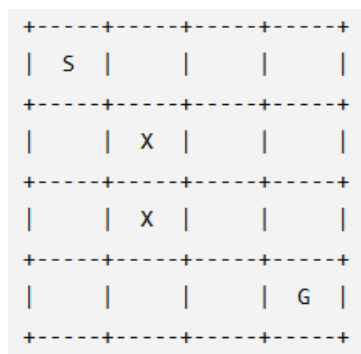
- Policy Gradient
- Actor-Critic
- PPO

Diagram

Draw this feedback loop:

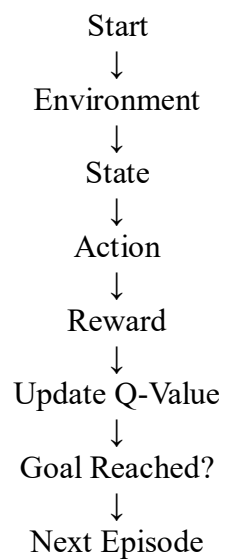


Navigation Diagram



- S = Start
- G = Goal
- X = Obstacle

Flowchart



↓
Evaluate

Evaluation

Use these measures:

- Average reward
- Success rate
- Steps required to reach goal
- Epsilon value
- Q-values
- Number of failed episodes

Conclusion

The RL agent may fail because of sparse rewards, poor exploration, incorrect reward design, incorrect state representation or unsuitable hyperparameters. Reward shaping, epsilon decay, Q-learning optimization and policy optimization can improve navigation performance.